

halfedge: wgpu 几何导出模块设计文档

src/export.rs

halfedge 库

2026-07-04

目录

1 模块定位	1
2 设计目标	2
2.1 为何需要单独的导出模块?	2
2.2 为何不用 <code>Vec<f32></code> 而用 <code>Vec<[f32; 3]></code> ?	2
3 导出算法	2
3.1 流程图	2
3.2 阶段 1: 顶点收集与索引映射	2
3.3 阶段 2: 索引生成	3
4 精度与兼容性	3
4.1 <code>f64</code> $\rightarrow$ <code>f32</code> 截断	3
4.2 wgpu 集成示例	3
5 复杂度	4
6 退化守卫	4
7 测试覆盖	4

1 模块定位

`export.rs` 提供单一函数 `mesh_to_vertex_index_buffers`: 将半边网格导出为 `Vec<[f32; 3]>` 顶点缓冲与 `Vec<u32>` 索引缓冲, 可直接传入 wgpu 创建 `Buffer`。

API	<code>mesh_to_vertex_index_buffers(&MeshStorage) -> (Vec<[f32; 3]>, Vec<u32>)</code>
-----	---

顶点缓冲 每个顶点 3 个 `f32` (位置 `x/y/z`)

索引缓冲 每个三角面 3 个 `u32` (CCW 朝向)

索引类型 `u32` (wgpu 默认 `IndexFormat::Uint32`)

顶点编号 按 `vertex_ids()` 顺序 0-based 重新编号

2 设计目标

2.1 为何需要单独的导出模块？

半边网格适合拓扑查询与编辑，但 `wgpu` 渲染需要扁平的顶点 + 索引缓冲：

- 顶点缓冲：连续内存，每个顶点固定字节数（此处 12 字节 = $3 \times \text{f32}$ ）；
- 索引缓冲：用顶点索引引用三角形，避免重复存储共享顶点。

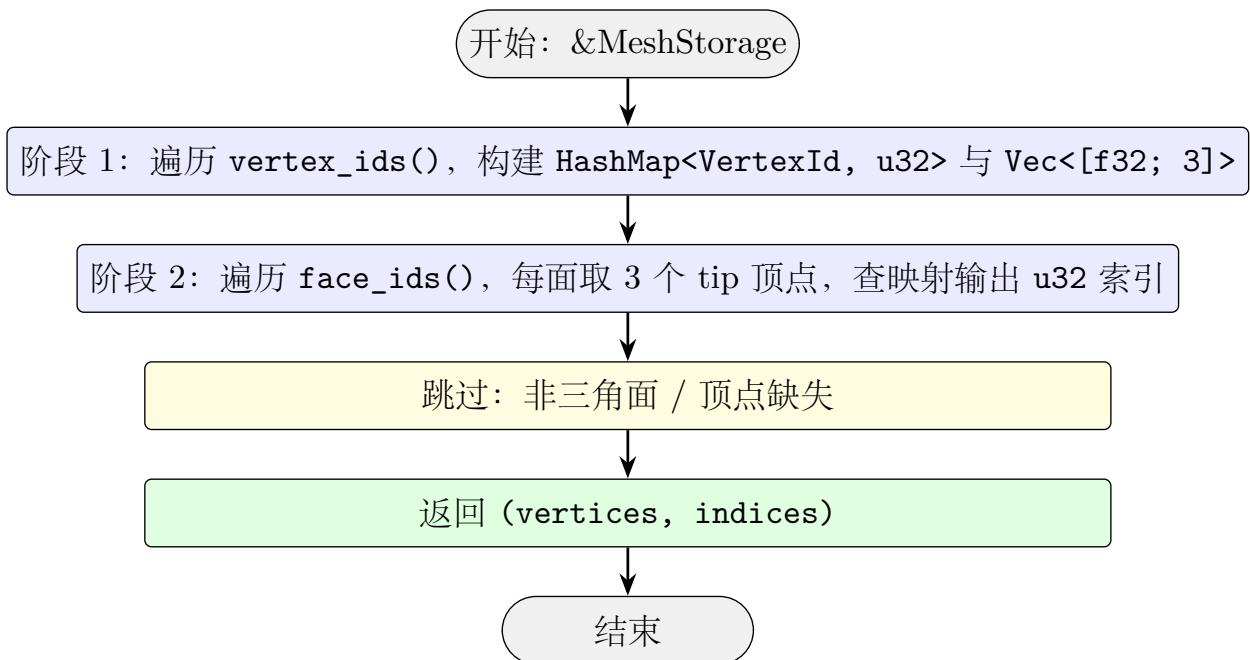
`export.rs` 负责「半边网格 → 扁平缓冲」的转换，是渲染管线的入口。

2.2 为何不用 `Vec<f32>` 而用 `Vec<[f32; 3]>`？

`[f32; 3]` 在内存中与 3 个连续 `f32` 完全等价（12 字节，4 字节对齐），但语义更清晰：每个元素代表一个顶点位置。调用方可直接 `as_ptr()` 转 `*const f32` 传给 `wgpu::Buffer`，也可方便地 `extend` 法向、UV 等属性。

3 导出算法

3.1 流程图



3.2 阶段 1：顶点收集与索引映射

- 遍历 `mesh.vertex_ids()`（slotmap 顺序）；
- 对每个 `VertexId`，取其位置 `[f64; 3]`，截断为 `[f32; 3]`；
- 用 `HashMap<VertexId, u32>` 记录「`VertexId` → 0-based 索引」映射；
- 顶点缺失（已删除）跳过。

3.3 阶段 2：索引生成

- 遍历 `mesh.face_ids()`;
- 对每个面，用 `FaceHalfEdges` 迭代器取边界环半边；
- 取每条半边的 `vertex` (`tip`)，查 `HashMap` 得 0-based 索引；
- 边界环长度 $\neq 3$ 跳过（非三角面）；
- 索引按 CCW 顺序输出（与原面 `next` 链一致）。

4 精度与兼容性

4.1 f64 $\rightarrow$ f32 截断

半边网格内部用 `f64` 存储顶点位置（精度高，适合几何计算）；导出时截断为 `f32`（GPU 原生支持，性能优）。

精度损失：`f64` 有约 15–17 位有效数字，`f32` 约 6–9 位。对大多数渲染场景（屏幕分辨率、视锥范围）足够；若需要高精度渲染（如 CAD 模型远距离观察），可扩展为 `Vec<f32; 6>`（双精度拆分）或使用 `wgpu::VertexAttribute` 的 `Float64` 格式。

4.2 wgpu 集成示例

```
1 use wgpu::util::DeviceExt;
2
3 let (vertices, indices) = mesh_to_vertex_index_buffers(&mesh);
4
5 let vertex_buffer = device.create_buffer_init(
6     &wgpu::util::BufferInitDescriptor {
7         label: Some("Vertex Buffer"),
8         contents: bytemuck::cast_slice(&vertices),
9         usage: wgpu::BufferUsages::VERTEX,
10    },
11 );
12
13 let index_buffer = device.create_buffer_init(
14     &wgpu::util::BufferInitDescriptor {
15         label: Some("Index Buffer"),
16         contents: bytemuck::cast_slice(&indices),
17         usage: wgpu::BufferUsages::INDEX,
18    },
19 );
20
21 //
22 // render_pass.set_vertex_buffer(0, vertex_buffer.slice(..));
23 // render_pass.set_index_buffer(index_buffer.slice(..), wgpu::IndexFormat::Uint32);
24 // render_pass.draw_indexed(0..indices.len() as u32, 0, 0..1);
```

5 复杂度

$O(V + F)$: 每个顶点与每个面常数时间。

阶段	时间	空间
顶点收集	$O(V)$	$O(V)$ (<code>Vec</code> + <code>HashMap</code>)
索引生成	$O(F)$	$O(F)$ (<code>Vec<u32></code>)
总计	$O(V + F)$	$O(V + F)$

6 退化守卫

- 顶点缺失 (`get_vertex` 返回 `None`): 跳过, 不输出;
- 面边界环长度 $\neq 3$: 跳过该面, 不输出索引;
- 半边 `vertex` 在 `HashMap` 中找不到 (理论不应发生, 因阶段 1 已收集所有顶点): `filter_map` 静默跳过。

7 测试覆盖

测试名	验证点
<code>export_basic_quad</code>	4 顶点 6 索引, 位置正确
<code>export_tetrahedron</code>	4 顶点 12 索引 (4 面)
<code>export_preserves_ccw_winding</code>	导出索引 CCW 朝向, 法向 $+z$
<code>export_empty_mesh</code>	空网格输出空缓冲
<code>export_roundtrip_validation</code>	导出 $\rightarrow$ 重建 $\rightarrow$ 通过校验